DESIGNING DATA-INTENSIVE APPLICATIONS · VISUAL COMPANION

Chapter 3
Storage and Retrieval

How databases store bytes on disk — and why LSM-trees, B-trees, and columnar engines make different bets

Source: claude-skills / ch03-storage-and-retrieval.md · Illustrated · Interview-ready

Martin Kleppmann · Part I: Foundations

Table of Contents

1. Core Idea

Storage engines split along two axes:

📝 Log-structured

Append-only, never modify in place — Bitcask, LSM-trees, LevelDB, RocksDB, Cassandra, HBase, Lucene.

✏️ Update-in-place

Fixed-size pages overwritten on disk — B-trees, standard in virtually every relational database.

⚖️ Universal trade

Well-chosen indexes speed up reads, but **every index slows down writes** — choose from query patterns.

Workload	Bottleneck	Winning engine
OLTP — many small key lookups	Disk *seek time*	Row-oriented + indexes (B-tree / LSM)
OLAP — few huge scans	Disk *bandwidth*	Column-oriented + compression

Sequential beats random. Log-structured engines win writes by turning random writes into sequential ones. Indexes trade write speed for read speed — never add them by default.

2. Framework: LSM-tree (Log-Structured Merge-Tree)

Write-heavy workloads needing high random-write throughput. LevelDB, RocksDB, Cassandra, HBase — structure from O'Neil's LSM-tree paper, terms from Google's Bigtable.

When	Write-heavy workloads; high random-write throughput without seek-bound I/O.
How	Writes go to an in-memory balanced tree (**memtable**); past a threshold (few MB) flushed to disk as an **SSTable** (Sorted String Table — key-sorted, each key once per segment). Reads check memtable, then segments newest-to-oldest. Background **compaction** merges segments mergesort-style, keeping newest value per key and honoring tombstones. Separate unsorted append-only log restores memtable after crash.
Why it works	Systematically turns random writes into sequential writes — faster on spinning disks and SSDs. Sorting enables sparse in-memory indexes (one key per few KB), efficient merging, block compression, and range queries.
Failure mode	Lookups of nonexistent keys check every segment — fixed with **Bloom filters** (memory-efficient set approximation). Compaction competes with live requests → high-percentile latency spikes.

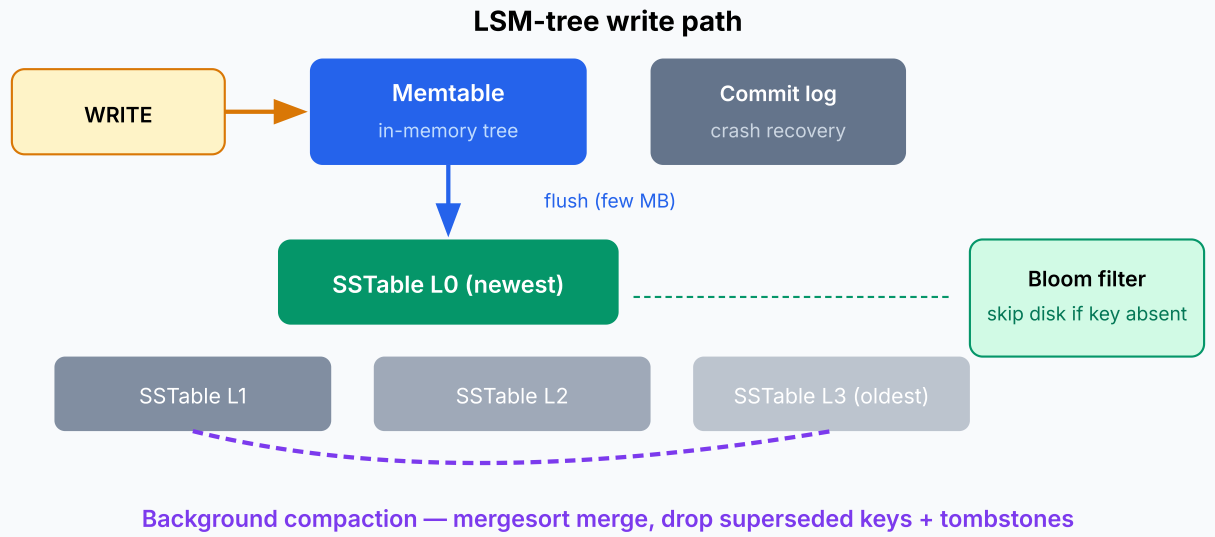

Figure 2.1 — LSM: memtable + commit log → SSTables → compaction. Bloom filters skip absent keys.

Real systems: LevelDB/RocksDB (embedded KV), Cassandra/HBase (distributed), Lucene (inverted index segments). Columnar warehouses reuse the same LSM idea for write paths (Vertica).

3. Framework: B-tree

Default choice. Read-heavy or mixed workloads; anywhere strong transactional semantics matter. PostgreSQL, MySQL InnoDB, SQL Server.

When	Default; read-heavy or mixed workloads; strong transactional semantics.
How	Database broken into fixed-size pages (traditionally 4 KB), forming a balanced tree. Each page holds k keys and k+1 child references (branching factor in hundreds), height O(log n). Updates overwrite the leaf page in place; full page splits into two half-full pages and parent is updated.
Why it works	Pages match hardware block structure. Every key exists in exactly one place — range locks for transaction isolation attach directly to the tree.
Failure mode	Crash mid-split corrupts index (orphan pages) — hence **write-ahead log (WAL/redo log)**: every modification appended to WAL before touching pages → every write hits disk at least twice. In-place mutation requires latches for concurrent access.

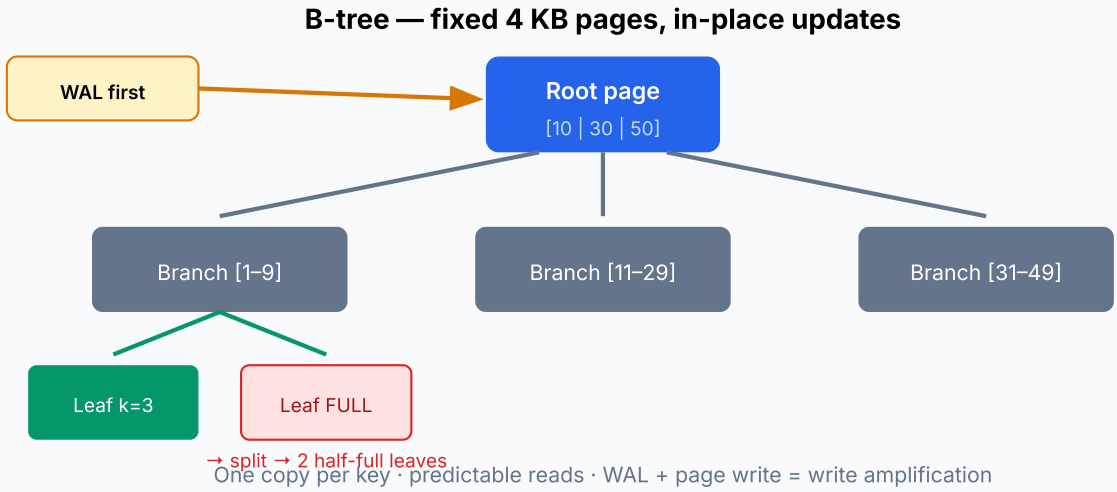

Figure 3.1 — B-tree: pages match disk blocks; WAL before in-place update; splits on overflow.

4. LSM vs B-tree Trade-offs

Dimension	LSM-tree	B-tree
Write path	Sequential append + background compaction	WAL + in-place page overwrite (+ splits)
Read path	Check memtable + multiple SSTables (Bloom filters help)	Single tree traversal — one copy per key
Predictability	Compaction pauses → tail latency spikes	Stable; no background rewrite storms
Transactions	Harder — keys scattered across segments	Range locks attach directly to tree
Write amplification	Repeated rewriting during compaction	WAL + page (+ splits) — costly on SSD wear

Rule of thumb: LSM faster for writes; B-tree thought faster for reads and more predictable. Kleppmann's verdict: *neither side clearly wins — benchmark your own workload.* There is no quick and easy rule.

Write amplification = one logical write → multiple physical disk writes. Especially costly on SSDs (limited overwrite cycles). Both engines pay it; the mechanism differs.

5. Indexing Patterns

Pattern	How	Trade-off
Secondary index	Keys not unique; store list of row IDs per key (postings list) or append row ID to make keys unique	Extra write on every indexed column change
Heap file	Rows stored unordered; indexes hold references	Avoids duplication across indexes
Clustered index	Row stored inside the index (InnoDB primary key); secondary indexes point at primary key	Faster primary-key lookups; more storage + write overhead
Covering index	Some columns stored in the index — query answered from index alone	Read speed vs storage + write cost
Concatenated index	Phone-book ordering (lastname, firstname) — great for prefix queries	Useless for second column alone; 2D geo needs R-trees or space-filling curves

🖨️ **Index-cost lens:** every index is a read/write trade. Ask "what query pattern pays for this index?" before adding it. Indexing everything "just in case" is pure write overhead.

6. OLTP vs OLAP

Dimension	OLTP	OLAP
Query pattern	Small number of records per query, fetched by key	Aggregates over huge scans
Writes	Random, low-latency	Bulk ETL loads
Users	End users (apps)	Internal analysts
Data focus	Latest state	History of events
Scale	GB–TB	TB–PB
Bottleneck	Seek time → indexes matter	Bandwidth → compression matters

Ad-hoc analytics on production OLTP — huge scans wreck latency for concurrent transactions. The whole reason data warehouses exist.

7. Data Warehouse & Star Schema

Read-only copy of all OLTP systems, loaded via **ETL** (Extract-Transform-Load).

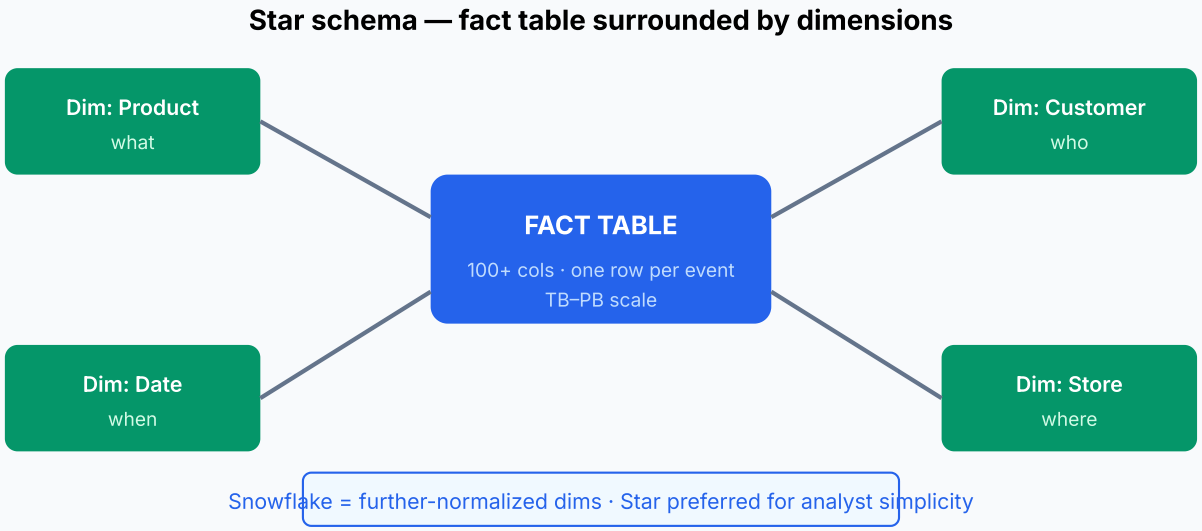

Figure 7.1 — Star schema: denormalized dimensions around a wide fact table.

8. Column-Oriented Storage

Store each column's values in its own file, same row order in every file. Query touching 4–5 of 100+ columns reads only those files.

Technique	What it does
Column files	Read only columns needed — bandwidth win on wide fact tables
Bitmap encoding	Per distinct value, a bitmap of matching rows
Run-length encoding	Compress repeated values in sorted columns
Vectorized processing	Tight loops over L1-cache-sized compressed chunks; bitwise AND/OR on bitmaps
Write path	Can't insert into compressed sorted columns in place — buffer in memory, bulk-merge (LSM idea; Vertica)

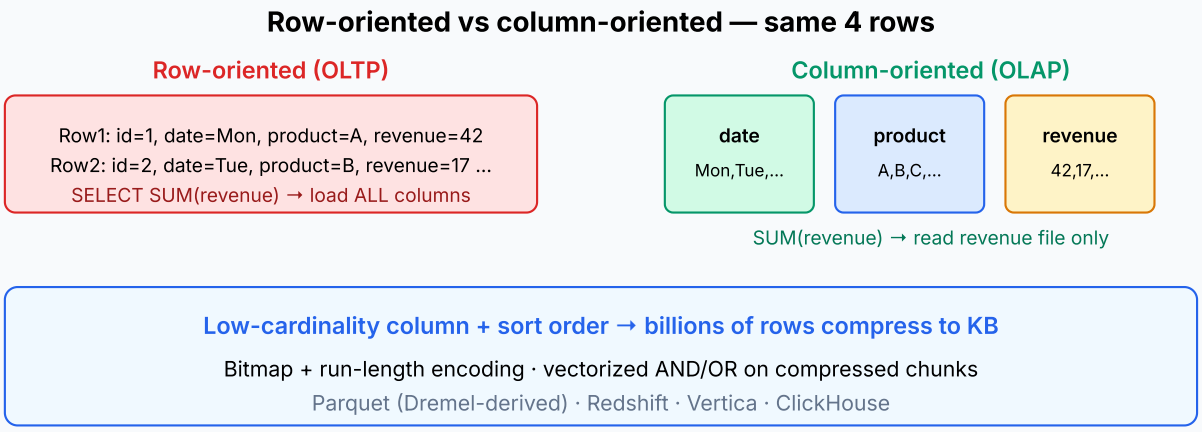

Figure 8.1 — Columnar: scan bandwidth, not seek time. Writes reuse LSM buffering.

9. Materialized Views & Data Cubes

Materialized views / data cubes: precomputed aggregate grids (SUM by date × product × ...) make specific queries instant but lose flexibility — can't query a non-dimension attribute.

> Warehouses keep raw data and use cubes only as a boost. Don't cube everything upfront.

10. Mental Models

1. Append-only logs

Sequential I/O beats random I/O on every medium. Immutable segments make concurrency and recovery trivial — no half-overwritten values.

2. Index-cost lens

Every index is a read/write trade. Ask "what query pattern pays for this index?" before adding it.

3. Seek vs bandwidth

Bottleneck is finding records (seeks) → OLTP engine, indexes matter. Moving bytes (bandwidth) → columnar, compression matters.

4. LSM vs B-tree default

B-trees when predictability and transactions dominate; LSM when write throughput dominates — then verify empirically.

11. Anti-Patterns

Anti-pattern	Why it fails
Ad-hoc analytics on production OLTP	Huge scans wreck concurrent transaction latency
Hash indexes when keys don't fit in RAM	Bitcask constraint; no range queries (degenerates to one lookup per key)
Row-oriented storage for wide fact tables	Loading 100+ attribute rows to use 3 columns burns disk bandwidth
Indexing everything "just in case"	Pure write overhead with no query paying for it
Treating LSM-vs-B-tree benchmarks as portable	Results are workload- and tuning-sensitive; test with *your* workload

12. Worked Example: db_set → LSM Ladder

Start with the world's simplest database — two bash functions:

```
db_set () { echo "$1,$2" >> database; }
db_get () { grep "^$1," database | sed -e "s/^$1,//" | tail -n 1; }
```

Writes are O(1) appends (it's a log); reads are O(n) scans.

0. **Append-only log** — writes O(1), reads O(n) full scan
1. **Hash index (Bitcask)** — in-memory hash map key → byte offset; reads become one seek. **Compaction:** keep newest value per key, merge segments in background, atomically swap. Limits: all keys in RAM; no range queries.
2. **SSTables** — require segments sorted by key. Merging = streaming mergesort (works when files exceed memory; newer segment wins). Index goes sparse (one key per few KB). Blocks compress.
3. **LSM-tree** — buffer unsorted writes in memtable, flush to SSTable at few MB, crash-recovery log, background compaction, Bloom filters for missing keys. LevelDB/RocksDB/Cassandra/HBase.

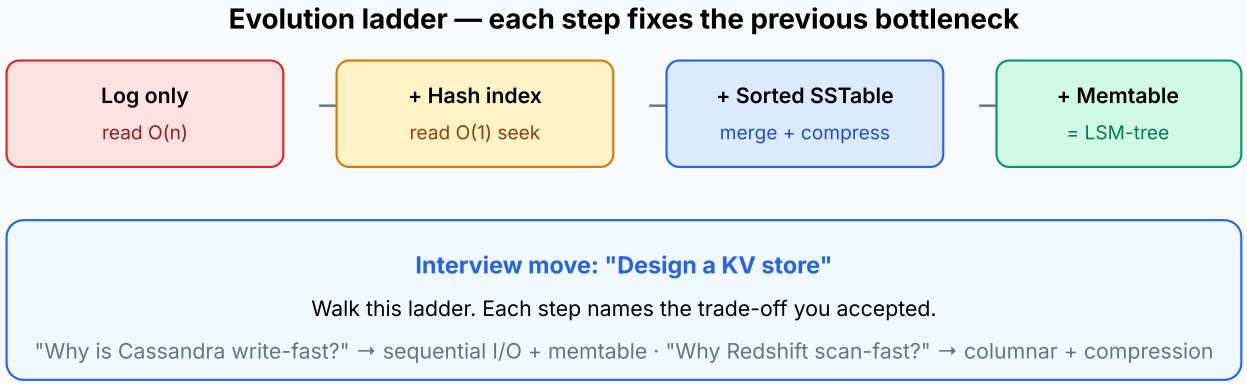

Figure 12.1 — db_set → hash index → SSTable → LSM: the canonical KV-store design interview ladder.

13. Problems Each Mechanism Solves

Mechanism	Problem it solves	Example
LSM write path (commit log → memtable → SSTable) + Bloom filter reads	Write-heavy key-value workload needs high throughput without random-write cost	Leaderless key-value store
Bloom filter as existence check before point lookup	Avoid wasted lookup on huge sparse key space	URL shortener (`shortURL → longURL`)
In-memory index rebuilt at startup, no incremental persistence	Structure only makes sense in memory; rebuild is cheap, incremental update isn't	Proximity service (quadtree over live location data)
Purpose-built time-series engine: delta-of-delta encoding + downsampling	Metric points are mostly-monotonic timestamps with tiny deltas — generic storage wastes bytes	Metrics monitoring & alerting
Sorted set (hashmap + skip list) for O(log n) rank	Relational nested-count query for "what's my rank" doesn't scale past one table	Real-time leaderboard
LSM justification for custom write-heavy index	Sequential-write engine needed for bespoke search structure	Distributed email service (custom search engine)
mmap append log + embedded LSM (RocksDB) + periodic snapshots	Need both durable sequential log and fast point reads on same node	Digital wallet
WAL segments exploiting sequential disk + OS page cache	Broker must sustain high write throughput without seek-bound I/O	Distributed message queue

14. Connects To (Cross-Chapter)

Chapter	Connection	Interview soundbite
Ch 2 Data Models	Same question from the database's side — Ch 2 was how you hand data over; Ch 3 is how the engine stores and finds it.	"Document vs relational is the API; LSM vs B-tree is the engine."
Ch 4 Encoding	On-disk formats (SSTable layouts, Parquet's Dremel-derived columnar format) are encoding decisions.	"Storage format = encoding + indexing strategy."
Ch 7 Transactions	B-trees' one-place-per-key property lets range locks attach to the tree; copy-on-write B-trees (LMDB) feed into snapshot isolation.	"WAL underpins atomicity and durability."
Ch 10–11 Batch/Stream	ETL is batch processing; the append-only log reappears as the backbone of stream systems.	"The log is the universal primitive — KV store to Kafka."

15. Key Takeaways

1. **Sequential beats random:** log-structured engines win writes by turning random writes into sequential ones.
2. **Indexes trade write speed for read speed** — choose them from query patterns, never by default.
3. **B-tree** = one copy per key, in-place updates, WAL, predictable, transaction-friendly. **LSM** = append-only, compaction, higher write throughput, compaction-induced tail latency. Both suffer write amplification; benchmark to decide.
4. **OLTP is seek-bound** (index by key); **OLAP is bandwidth-bound** (scan compactly). Separate the systems.
5. **Columnar storage** + bitmap/run-length compression + sort order is the OLAP answer; its write path reuses the LSM idea.
6. **Star schema** (fact + dimension tables) is the near-universal analytics model; ETL feeds it. Keep raw data; cubes are only a boost.

16. Junior SWE Checkpoints

Q1: Why does every index slow down writes?

Every write must update every index that covers the changed columns. Indexes are read accelerators paid for at write time — choose them from query patterns, not by default.

Q2: "Design a key-value store" — what's the ladder?

Append-only log (write O(1), read O(n)) → hash index/Bitcask (read = one seek, keys in RAM) → sorted SSTables (merge + sparse index + compression) → LSM-tree (memtable + flush + compaction + Bloom filters). Name the trade-off at each step.

Q3: Why is Cassandra write-fast but read paths can spike?

LSM: sequential writes via memtable + SSTable flush. Reads may check multiple SSTables; compaction competes with live traffic → tail latency spikes. Bloom filters help absent-key lookups.

Q4: Why does a B-tree write hit disk at least twice?

WAL append first (durability), then in-place page write. Page splits add more writes — that's write amplification. Crash mid-split is recovered from WAL.

Q5: OLTP vs OLAP — what's the bottleneck difference?

OLTP: seek time → row storage + indexes. OLAP: bandwidth → columnar files + compression. Running analytics on OLTP destroys concurrent transaction latency.

Q6: Why can't you insert directly into compressed column files?

Sorted, compressed columns can't accept in-place inserts. Fix: buffer in memory, bulk-merge into new column files — the LSM idea (Vertica does this).

Q7: How does Ch 3 connect to transactions (Ch 7)?

B-tree's one-copy-per-key lets range locks attach to the tree. WAL provides atomicity/durability. Copy-on-write B-trees (LMDB) enable snapshot isolation without in-place mutation.

Glossary

Term	Definition
Memtable	In-memory balanced tree buffering recent writes before SSTable flush
SSTable	Sorted String Table — immutable on-disk segment, key-sorted, one value per key
Compaction	Background mergesort merge of SSTables; drops superseded keys, honors tombstones
Bloom filter	Probabilistic set: can say "definitely not present" — skips disk reads
WAL	Write-ahead log — append modification before touching pages
Write amplification	One logical write → multiple physical disk writes
Clustered index	Row stored inside index (InnoDB PK); secondaries point at PK
Covering index	Index stores enough columns to answer query without heap lookup
ETL	Extract-Transform-Load — pipeline from OLTP to warehouse
Star schema	Wide fact table surrounded by denormalized dimension tables
Columnar storage	Each column in its own file; same row order across files
Data cube	Precomputed aggregate grid — fast for fixed dimensions, inflexible otherwise